2022 年同等学力申硕计算机综合试题-软件工程

一、单项选择题 (每空 1 分，共 5 分)

软件开发模型是指软件开发的全部过程、活动和任务的结构框架。主要的开发模型有：瀑布模型、演化模型、螺旋模型、喷泉模型等。螺旋模型将瀑布模型和演化模型相结合，并增加了(1)，它建立在(2)的基础上，沿螺旋自内向外每旋转一圈，就得到它的一个新版本。喷泉模型描述了(3)的开发模型，它体现了这种开发方法创建软件过程所固有的(4)和(5)的特征。

供选择的答案：

(1)：A. 系统工程 B. 风险分析 C. 设计评审 D. 进度控制

(2)：A. 模块划分 B. 子程序分解 C. 设计 D. 原型

(3)：A. 面向对象 B. 面向数据流 C. 面向数据结构 D. 面向事件驱动

(4)：A. 归纳 B. 推理 C. 迭代 D. 递归

(5)：A. 开发各阶段之间无间隙 B. 开发各阶段分界明显 C. 部分开发阶段分界明显 D. 开发过程不分段

解析：考察软件过程

(1) B. 风险分析(2) D. 原型(3) A. 面向对象(4) C. 迭代(5) A. 开发各阶段之间无间隙

- **螺旋模型的核心补充（第 1 空）：**螺旋模型在瀑布模型（线性阶段）和演化模型（迭代开发）的基础上，增加了**风险分析**环节。每轮螺旋开发都需评估技术、成本、进度等风险，通过风险驱动决策，因此 (1) 选 B。
- **螺旋模型的基础（第 2 空）：**螺旋模型的开发过程基于**原型迭代**：从核心功能原型出发，每旋转一圈（完成一次迭代）都完善原型，逐步逼近最终产品，因此 (2) 选 D。
- **喷泉模型的适用方法（第 3 空）：**喷泉模型是**面向对象开发**的典型模型，其名称源于面向对象方法中 “分析、设计、实现” 阶段的无缝衔接，如同喷泉的水循环，因此 (3) 选 A。
- **喷泉模型的过程特征（第 4 空）：**面向对象开发中，分析、设计、编码等活动常**迭代进行**（如先开发部分类，测试后再扩展），喷泉模型体现了这种迭代性，因此 (4) 选 C。
- **喷泉模型的阶段特征（第 5 空）：**与瀑布模型的严格阶段划分不同，喷泉模型中**各开发阶段无明显间隙**，分析、设计、实现可交叉进行（如边分析边设计），因此 (5) 选 A。

软件过程模型	模型描述
瀑布模型	将软件生存周期的各项活动规定为依固定顺序而连接的若干阶段工作，规定了每一阶段的输入，以及本阶段的工作成果，并作为输出传入下一阶段。
增量模型	该模型有一个假设，即需求可以分组，形成一个一个的增量；在这一条件下，可对每一增量实施瀑布式开发。
演化模型	是一种有弹性的过程模式，由一些小的开发步组成，每一步历经需求分析、设计、实现和验证，产生软件产品的一个增量，通过这些迭代，

淘宝店：DaMaCode编程文档源码库，原创

	完成最终软件产品的开发。
螺旋模型	该模型将软件生存周期的活动分为四个可重复的阶段：规划、风险分析、开发和评估。
喷泉模型	该模型体现了软件创建所固有的迭代和无间隙的特征。

二、判断题 (每小题 1 分，共 5 分。如果正确，用“√”表示；否则，用“×”表示)

1. 对象把数据和处理数据的操作结合为一体。（√）

解析：考察软件设计-面向对象方法

面向对象思想的核心特征之一是“封装”，即对象将其内部的数据（属性）和对数据的操作（方法）封装为一个整体，外部只能通过对象提供的接口（方法）访问或修改数据，而无法直接操作内部数据。这种“数据 + 操作”的结合形式，是面向对象方法区别于结构化方法中“数据与操作分离”的关键特征。因此，题干说法正确。

2. 程序是一些对象的集合，在程序运行时对象都处于活动状态。（×）

解析：考察软件设计-面向对象方法

题干存在两处关键偏差：

- **程序的构成表述不准确：**面向对象程序的核心是“对象的交互”，但程序不仅包含对象，还需通过对象间的消息传递（如调用方法）实现功能，并非单纯“对象的集合”；且非面向对象程序（如结构化程序）根本不基于对象构建，更不存在“对象集合”的说法。
- **对象活动状态表述错误：**程序运行时，并非所有对象都处于活动状态。部分对象可能仅在特定场景下被创建（如用户触发操作时），或创建后因完成任务被销毁，存在“未创建”“已销毁”等非活动状态，并非全程活动。

因此，题干说法错误。

3. 用计数方法而不是用文件结束符或输入序列结束符来判断程序输入的结束。（×）

解析：考察软件设计

程序判断输入结束的方式需根据场景选择，不存在“必须用计数方法”的固定规则，核心问题在于题干表述的“绝对性”：

- **计数方法的局限性：**仅适用于“输入数据量已知”的场景（如明确需输入 10 个学生成绩），若输入数据量未知（如用户随机输入文本直到主动停止），计数方法完全无法适用。
- **其他合法结束方式的必要性：**实际开发中，文件结束符（如 C 语言的 EOF）、输入序列结束符（如用户输入特定字符“#”）是更常用的结束判断方式，尤其适用于数据量不确定的场景，这些方式与计数方法同等合法，无优劣之分。

因此，题干中“用计数方法而不是用其他方式”的绝对化表述错误，输入结束

淘宝店：Pandaoo 懒文档源码店 原创

判断需结合具体需求选择合适方式。

4. 在面向对象系统中，类与对象的关系是抽象与具体的关系。（√）

解析：考察软件设计-面向对象方法

在面向对象系统中：

- **类（Class）**：是对一类具有相同属性（数据）和方法（操作）的对象的抽象描述，是一个概念性的模板，不直接对应具体实体（如 “学生” 类是对所有学生的抽象）。
- **对象（Object）**：是类的实例化结果，是具体存在的实体，具备类所定义的属性和方法（如 “张三” 是 “学生” 类的一个具体对象）。

因此，类是对对象的抽象概括，对象是类的具体体现，二者是抽象与具体的关系，题干说法正确。

5. 接受消息的对象必须回答消息发送者。（×）

解析：考察软件设计-面向对象方法

面向对象系统中，消息传递是对象交互的核心方式，但 “接受消息的对象必须回答发送者” 的表述不符合实际逻辑，关键原因如下：

- **消息类型决定是否需 “回答”**：消息分为 “请求型消息”（如调用对象的查询方法，需返回结果，如 “获取用户年龄”）和 “通知型消息”（仅告知对象执行操作，无需返回结果，如 “弹出提示框” “记录日志”）。对于通知型消息，接收对象执行操作后无需向发送者反馈，即无需 “回答”。
- **“回答” 并非强制规则**：消息传递的核心是 “触发操作”，而非 “强制反馈”。是否需要返回结果，取决于业务逻辑设计（如仅需执行动作时，无需返回），不存在 “必须回答” 的强制要求。

因此，题干中 “必须回答” 的绝对化表述错误。

- **总结面向对象核心概念（类、对象、消息）关系：**
  - **类**是 “抽象模板”，**对象**是 “具体实例”，二者是 “抽象与具体” 的关系；
  - **消息**是对象之间的 “通信方式”，依赖类定义的方法实现交互，是面向对象系统中功能协作的核心纽带。

三、简答题（每小题 4 分，共 12 分）

1. 以 G.J.Myers 的观点，简述软件测试的目的。

解析：考察软件测试

一、G.J.Myers 关于软件测试目的的核心观点

G.J.Myers 在《软件测试艺术》中提出了与传统 “找错” 认知不同的测试目的，核心可概括为：**软件测试的目的是 “以最少的代价，在最短的时间内，找出软件中尽可能多的错误”，同时验证软件是否满足用户需求**，而非单纯证明软件无错。其具体观点可拆解为 4 点：

二、软件测试目的的具体阐述

- **(1)首要目的：发现错误**

  Myers 认为，测试的核心价值在于主动暴露软件中的缺陷（如功能异常、逻

淘宝店：PanJaCode编程学习源码库，原创

辑漏洞、边界错误），而非验证软件 “能正常工作”。只有发现错误，才能通过修复提升软件质量，避免错误流入用户手中导致更大损失。

- **(2)次要目的：验证需求满足度**
  测试需对照需求规格说明书，验证软件功能、性能、安全性等是否符合用户预期。例如，通过测试确认 “用户登录模块” 是否满足 “支持手机号 / 邮箱两种登录方式” 的需求，确保软件方向与用户需求一致。
- **(3)辅助目的：评估软件质量**
  测试过程中收集的错误数据（如错误类型、出现频率、修复成本）可用于评估软件质量水平。例如，若测试中发现 “支付模块” 存在高频率转账计算错误，可判断该模块质量风险较高，需重点优化。
- **(4)最终目的：降低风险与代价**
  测试需在软件发布前发现并修复错误，避免错误在生产环境中爆发（如电商平台支付漏洞导致用户财产损失）。通过 “早期测试”（如单元测试、集成测试）减少后期修复成本，实现 “以最少代价保障质量” 的目标。

表4-12 软件测试的基本概念

软件测试	基本概念
定义	测试是为了发现程序中的错误而执行程序的过程，其不能证明程序中没有错误
目标	①好的测试方案是极可能发现迄今为止尚未发现的错误的测试方案； ②成功的测试是发现了至今为止尚未发现的错误的测试； ③根本目标是尽可能多地发现并排除软件中潜藏的错误，最终把一个高质量的软件系统交给用户使用； ④测试的目标是发现错误，调试的任务是在测试的基础上去改正错误
测试方法	①白盒测试（结构测试）：把程序看成装在一个透明的白盒子里，测试者完全知道程序的结构和处理算法，按照程序内部的逻辑测试程序，检测程序中的主要执行通路是否都能按预定要求正确工作； ②黑盒测试（功能测试）：把程序看作一个黑盒子，完全不考虑程序的内部结构和处理过程，是在程序接口进行的测试，只检查程序功能是否能按照规格说明书的规定正常使用

2. 消息传递是面向对象方法中对象之间进行交互的机制，消息传递机制与传统程序设计模式中的过程调用有什么本质区别?

解析：考察软件设计-面向对象方法

消息传递机制与传统过程调用的本质区别

对比维度	面向对象的消息传递机制	传统程序设计的过程调用
调用主体与接收主体	主体是**对象**：消息由一个对象发送给另一个对象（接收对象），请求其执行特定方法（如 “对象 A 向对象 B 发送‘计算’消息”）。	主体是**过程 / 函数**：调用者直接指定过程名（如 sum(a,b)），接收者是独立的过程，与调用者无 “对象” 身份关联。
依赖关系	发送者仅需知道接收对象的**接口（可接收的消息类型）**，无需了解其内部实现（如方法如何编码），体现 “封装性”。	调用者需知道过程的**具体实现细节**（如参数类型、返回值、内部逻辑），否则无法正确调用，依赖紧密。
灵活性与动态性	接收对象可动态决定如何响应消息：同一消息被不同对象接收时，	过程调用是静态绑定：调用哪个过程由调用语句直接指定，

淘宝店：NandaCode文档源码库 原创

	可执行不同实现（多态性），如“‘绘图’消息对‘圆形’和‘矩形’对象有不同处理”。	执行逻辑固定，无动态适配能力。
参数传递的侧重点	消息包含**操作名称和参数**，核心是“请求对象执行操作”，参数是操作的输入，不直接暴露接收对象的内部数据。	过程调用传递的是**数据参数**，核心是“对数据进行处理”，过程可能直接操作传入的全局数据或共享变量。
本质目标	实现对象间的**协作**：通过消息驱动多个对象分工完成复杂功能，体现“面向对象的交互逻辑”。	实现**功能复用**：将重复逻辑封装为过程，通过调用减少代码冗余，体现“功能的模块化”。

两者的核心区别在于：

- 消息传递是**对象级的交互**，依赖接口而非实现，支持动态多态，体现面向对象的封装性和灵活性；
- 过程调用是**功能级的复用**，依赖具体实现，静态绑定，更侧重代码的模块化而非实体间的协作。

这种差异使得消息传递更适合描述复杂系统中实体（对象）的动态行为和关系，而过程调用更适合简单功能的抽象与复用。

3. 说明面向对象分析阶段所建立的对象模型、动态模型和功能模型之间的关系。

解析：考察软件设计-面向对象方法

一、三大模型的核心定位（基础区别）

首先明确各模型的核心职责，这是理解其关系的前提：

模型类型	核心定位	描述视角	关键产出物
对象模型	描述系统的**静态结构**	“是什么”（实体及关系）	类图（类、属性、方法、关联）
动态模型	描述系统的**动态行为**	“怎么做”（行为时序 / 状态）	状态图、序列图
功能模型	描述系统的**功能目标**	“做什么”（功能及数据流向）	用例图、数据流图

二、三大模型的核心关联逻辑

面向对象分析的对象模型、动态模型和功能模型，从静态、动态、功能三个维度共同描述系统，三者相互依赖、互补，构成完整分析模型，具体关系如下：

- **对象模型是基础**
  对象模型定义系统的静态结构（类、属性、关联），是另外两个模型的载体。动态模型描述的“行为主体”（如对象状态变化、消息传递），以及功能模型描述的“数据载体”（如功能输入输出的属性），均需基于对象模型中的类或对象，无对象模型则动态与功能模型无实体依托。
- **动态模型是行为桥梁**
  动态模型通过状态图、序列图描述对象的动态交互（如“用户支付订单”的消息传递时序），将对象模型的“静态结构”转化为可执行的“动态行为”，同时落地功能模型的“抽象功能”—— 例如功能模型的“支付”功能，需通过动态模型明确“验证余额→扣减金额→更新订单状态”的行为步骤，否则功能仅为抽象目标。
- **功能模型是需求导向**

淘宝店 PandaCode辉文档源码库，原创

功能模型定义系统需实现的功能目标(如用例图中的“订单管理”“支付”)，决定对象模型的设计范围（如需 “支付” 功能则设计 “订单” “支付账户” 类)，同时引导动态模型的行为重点(优先细化核心功能的交互逻辑)，确保对象与动态模型围绕用户需求构建，避免偏离目标。

四、建模题 (共 8 分)

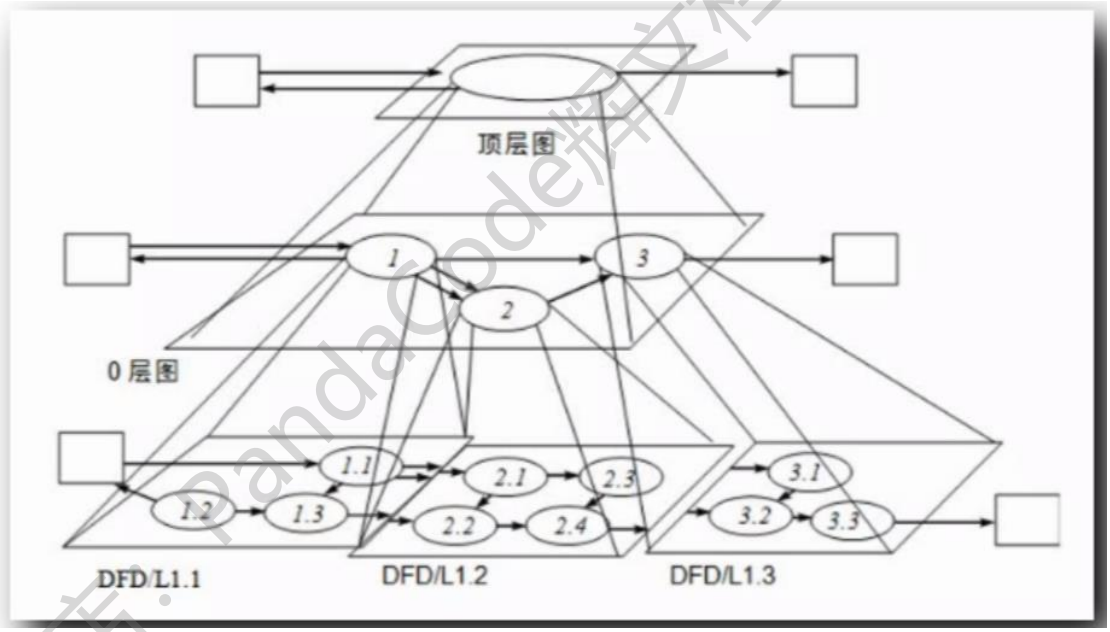
某培训中心要研制一个计算机管理系统，它的业务是将学员发来的信息收集分类后，按以下不同情况进行处理：

1，如果是报名，则将报名信息送给负责报名事务的职员，他们将查阅课程文件，检查该课程是否满额，然后在学生文件、课程文件、账目文件上登记，并开出报名单交财务部门，财务人员开出发票给学生；

2，如果是想注销原来已选修的课程，则由注销人员在课程文件、学生文件和账目文件上做相应的修改，并给学生注销单；

3，如果是付款，则由财务人员在账目文件上登记，也给学生一张收费收据。请根据该计算机管理系统的功能描述，采用“自顶向下”的原则，画出其分层数据流图（顶层 DFD 和 0 层 DFD)。

解析：考察软件设计-结构化方法



0、数据流图分层说明

数据流图分层，最上层是顶层数据流图，第二层是 0 层数据流图，最底层是底层数据流图，“顶层数据流图”与“底层数据流图”之间是若干中层数据流图，中层数据流图需要进行编号，从 0 开始编号。

结构化分析方法，简单说，从顶层简单到底层，逐层分解精细化。

1、顶层数据流图

顶层数据流图：中间的椭圆是需要开发的系统，周边的矩形表示的是外部实体人或组织，外部实体与系统之间，有数据传输关系；顶层数据流图能够表达的信息是非常有限的，其将整

淘宝店：PandaCode辉文档源码库 原创

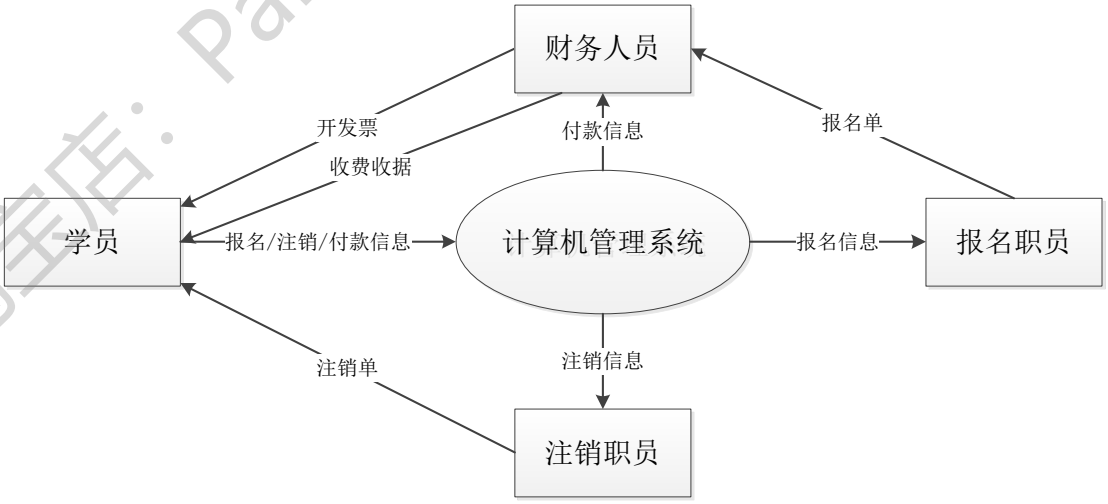
个系统，使用一个节点表示，其可以体现出系统与外界实体之间的交互，但是系统内部的情况，系统内部模块之间的数据交换是没有体现的。

2、中层数据流图(0 层)
将“顶层数据流图”进行细化，细化后的即 0 层数据流图；
与顶层数据流图比较，没有变化的部分：外部实体，外部实体与系统之间的数据流，是没有变化的；
变化部分：有变化的部分是系统内部，系统内部进行了细化，原来系统是一个节点，在中层数据流图(0 层)中，将一个节点拆分成多个节点，这些节点就是系统中的数据处理部件，即加工。这些数据处理部件（加工）之间会有数据流的交互。

3、底层数据流图(1 层)
针对每个加工节点，将其拆分，绘制其中的更详细的数据流转情况；
数据流图（DFD）分层，是从 顶层 -> 中层 -> 底层，逐层进行分解，这种分解思路，与结构化的开发方法，是完全匹配的。

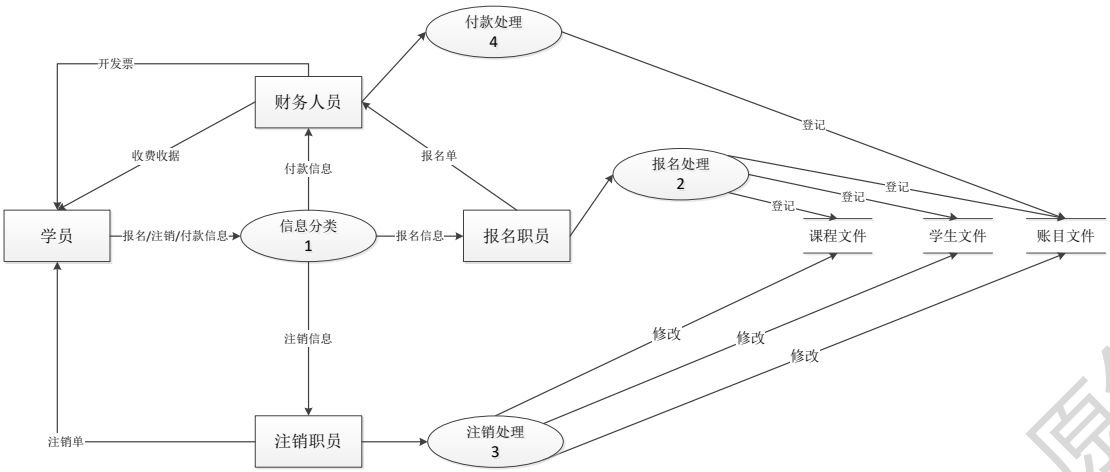
（1）顶层 DFD（系统环境图）如下：
在结构化分析中，顶层 DFD（数据流图）也被称为系统环境图。它用于展示系统与外部实体之间的数据流动。
在这个计算机管理系统中，我们可以识别出以下实体和数据流：

- **外部实体：**
  - 学员 – 报名/注销/付款信息。
  - 报名职员 – 报名单。
  - 注销职员 – 注销单。
  - 财务人员 – 开发票，收费收据。
- **数据存储：**
  - 学生文件 - 记录学生信息。
  - 课程文件 - 记录课程信息。
  - 账目文件 - 记录账目信息。



（2）0 层 DFD（数据流图）如下：（实际画图时数据流线尽量不能交叉！！！）

淘宝店：PanyCode辉文档源码库 原创



淘宝店：PandaCode辉文档源码库，原创